Working with Claude

Set up an AI assistant that remembers your work between sessions, files things for you, and checks its own output. For a book, a business, a website, a family history: anything.

The idea

Most people use AI as a chat window: you ask, it answers, and next week it has forgotten everything. This setup is different. You give Claude **its own folder on your computer**, like a filing cabinet, with one folder per project. In it, Claude keeps notes on your project, how you like things done, what you've decided and why, and a plan for next time. Each time you open the folder, Claude reads its notes first and picks up where you left off.

> **Is this for you?** It depends on the shape of the work. It suits work that runs over weeks or months and that you come back to in fits and starts, that lives in files on a computer, and that needs to be right. It's less suited to quick questions, shared online documents that others edit at the same time (Google Docs, Teams), work done mostly on a phone or tablet, and pupils', patients' or clients' details on a personal account. The Claude and ChatGPT chat apps both have **Projects** that keep your instructions and files with no setup beyond naming one; if that's all you need, start there, or use both. This kit adds memory you own (ordinary files in your folder, with no export step), a ready-made next step every time you come back, decisions and sources on record, and an assistant that files things for you and queues its work for your sign-off.

> **Each time you come back, once you're set up**
> 1. Open the Claude app → **Code** → **+ New session** → choose your project folder.
> 2. Say **hello**. Claude offers **one small next step**, already worked out. Say **"go"**.
> 3. Work by **talking**. Put new files in the **inbox** folder and say "process the inbox".
> 4. Stop whenever you like. Say **"that's it for today"** and Claude tidies up and saves, asking you nothing (for assessed work with a logbook, one question). Or just close the window: nothing's lost.

> **Happy to try a terminal?** It's the simplest way of all once it's installed: one window, no menus or buttons, and you just talk to it. Part 3 covers the terminal install (including Linux, and Gemini CLI or Codex instead of Claude); then come back for Step 3 (start it in your **Claude projects** folder and paste the sentence). Setup is the same: `/setup`. If your work is marked or assessed (a dissertation, coursework), read "Assessed work" in Part 2 before you start.

PART 1 · GETTING STARTED

Installing takes a little while. After that, setup takes about ten minutes and ends with a real piece of work done.

> **Use whichever you already have:**
>
> - **Claude** (Pro or above): the Claude app, or Claude Code in a terminal. This part of the guide follows the Claude app.
> - **ChatGPT** (Plus or above): Codex, in a terminal. See "Using Codex (ChatGPT) instead" in Part 3.
> - **A Google account** (free is fine): Gemini CLI, in a terminal. See "Using Gemini CLI instead" in Part 3.

What you need

- A Mac or Windows computer (Linux works too, in a terminal: see Part 3).
- For Claude: a **paid Claude plan** (Pro or above), a monthly subscription you can cancel. The free plan doesn't include what you need. Prices are at *claude.com/pricing*.
- Nothing else to download: in Step 3, Claude fetches the **starter kit** for you from *github.com/teedor/ai-workspace-kit*.
- **Using ChatGPT or Gemini?** Read its section in Part 3 first; Steps 3 and 4 are then the same, in a terminal.
- **Optional, for later:** two or three examples of your own work to show Claude your style, like an email you were pleased with, an old menu, a chapter or a letter.

Paid plans come with a usage allowance, shared between the Code tab and ordinary chats. If Claude says you've **reached your limit**, nothing is broken. It tells you when the allowance resets, and you won't be charged extra unless you've turned on **usage credits** in your account.

On the Pro plan, Claude Code starts on Opus, a top model that uses the allowance fastest. If you often reach your limit, Claude offers to switch your everyday model to Sonnet, which handles most work well and makes the allowance last longer, though it can be a little less thorough on the hardest problems. If you rarely reach your limit, you can ignore this.

STEP 1

Install the Claude app

1. Go to **claude.com/download**, download the app for your computer, and install it like any other app.
2. Open it and sign in.
3. **Privacy:** in the app's Settings, under Privacy, decide whether "Help improve Claude" should be on. Turn it off if you don't want your work used to train future models.

STEP 2

Git (Windows only; Mac users skip this)

Git is a free tool that saves snapshots of your work, so you can go back. Claude runs it for you.

- **Windows:** go to **git-scm.com**, download Git for Windows and install it. You'll see a dozen screens full of technical words. Ignore them all and keep clicking **Next**; the defaults are right. When it's finished, **close**

the Claude app completely and open it again, so it notices Git.

- **Mac:** nothing to do now. The first time Claude uses Git (in Step 3), a box may pop up offering to install "command line developer tools". Click **Install**, wait until it finishes (it can take a while), then tell Claude "done". If nothing seems to happen, look behind the Claude window.

STEP 3

Get the kit and make your first project folder

1. **Open your home folder.** Mac: in Finder, choose **Go → Home**. Windows: in File Explorer, click the address bar, type `%USERPROFILE%`, and press Enter. (Avoid Documents and Desktop. They're often synced to iCloud or OneDrive, and syncing can scramble the snapshots.)
2. Make a new folder there called **Claude projects**. All your projects will live in it. Leave it empty.
3. In the Claude app, click the **Code** tab at the top. Despite the name, it's for any kind of work.
4. Click **+ New session** in the left-hand list, or press `Cmd + N` (Mac) or `Ctrl + N` (Windows).
5. Around the box where you type, at the bottom, are a few settings. Set:
   - **Environment: Local** (it means "on this computer").
   - **Project folder** (it may say **Select folder**): click it and choose **Claude projects**. To reach your home folder in that window: on a Mac, press `Cmd + Shift + H`; on Windows, click the address bar at the top, type `%USERPROFILE%` and press Enter. If the app asks whether you trust this folder, say yes: it's your own.
   - **Permission mode: Manual** (older versions say *Ask permissions*). Claude then asks before every change, apart from its own notes.

   The app gets updated, so it may look slightly different, but those choices will be there.
6. Copy this sentence, paste it into the typing box, and press **Enter**. Paste it rather than dictating it: the web address has to be exact. (Line breaks don't matter.)

   ```
   Set me up with the AI Workspace Kit: clone github.com/teedor/ai-workspace-kit
   into ~/.claude/workspace-kit (on Windows use $HOME instead of ~), or update it
   if it's already there, deleting nothing. Then read its INSTALL.md (with the
   shell if your file tools can't reach it) and follow it. If Git is missing, stop
   and tell me how to install it.
   ```

7. Claude asks what your project is called, makes a **project folder** for it inside **Claude projects** (for example *Bakery* or *My novel*), and tells you how to open it.

Expect several permission requests, and Claude says what each one is for. The first saves the kit in Claude's own settings folder (`.claude` in your home folder); Claude always asks before writing there. Choose the plain **Yes** each time, not an option that stops it asking. If Claude gives you commands to type yourself, you're in the **Chat** tab: switch to **Code** and start again.

Only ever paste a "fetch this and follow it" sentence from somewhere you trust, like this guide. Claude follows what it finds there, within the permissions you give it.

> **Can't download the kit?** Some work laptops block it, and if a GitHub sign-in box pops up, close it: the kit doesn't need an account. Instead, go to *github.com/teedor/ai-workspace-kit*, click the green **Code** button → **Download ZIP**, and unzip it where it lands, in Downloads (Mac: double-click it; Windows: **right-click → Extract All… → Extract**). Then paste this instead: *"Set me up with the AI Workspace Kit from the ZIP I've unzipped in my Downloads folder: move the kit folder (the one with INSTALL.md in it) to ~/.claude/workspace-kit (on Windows use $HOME instead of ~), deleting nothing, then read its INSTALL.md and follow it."*

STEP 4

Your first session

Claude works on **one project folder at a time**, and you choose it at the start of each session. No rush, and you can stop halfway: setup picks up where it left off.

1. Start a **+ New session** as in Step 3, with the same settings, but this time choose your **project folder** (*Bakery*), not *Claude projects*. (It has to be a new session: Claude only reads a project's notes when the session is opened on that folder.)
2. Type `hello` and press **Enter**.
3. **Check the reply.** It should start by naming your folder, like *"📁 You're in Bakery."* If it doesn't, ask *"Which folder are you working in?"* If the answer isn't your project folder, start a new session and choose again.
4. Type `/setup` and press **Enter**. (Typing `/` pops up a menu; ignore it and keep typing.)

> **How to tell you've got the right folder:** your project folder has a file called `CLAUDE.md` and a folder called `inbox` directly inside it.

During setup

Claude asks a handful of questions, one at a time. Answer by talking or typing. It notes which computer you're on (so every instruction it gives fits your Mac or PC), sets up the snapshots (asking, if none is set yet, for a name and email to label them), then asks:

- to confirm the project's **name** (it suggests the folder name), what it is, and whether it involves code;
- whether it's **new**, or something you've been working on for a while;
- whether it's **marked or assessed work** (coursework, a dissertation) or has rules about using AI. If it's study or assessment, Claude adds the study pack (see Part 2); other rules are saved as they are;
- whether you'll put in **real people's details or data**: emails, orders, family records, datasets about people, recordings, or anything under a licence, contract or ethics approval. Those all count, so say yes if you'll use any of them;
- for **one small thing** you could do on it right now. Then it helps you do it. If you've already started, that thing is **catching up**: copy the three to five files that matter most into the inbox (right-click → Copy, then Paste, so your originals stay put), and Claude files them and writes up where things stand for you to check. The rest can come in later: say "catch up on this".

Everything else (a check of how the setup fits your work, a file about you that all your projects share, a style guide from your examples, dictation, Word, default critics for `/red-team`, and on Pro a lighter everyday model) is offered **later, one thing at a time**, at the end of a session that went well.

Expect permission requests, some in technical words. A few are for things outside your project folder, such as saving your name for the snapshots. **Claude tells you before each of those that it's coming and why.** Allow them.

> **You're set up when** Claude says *"You're set up."* If something goes wrong partway through, say *"something went wrong, explain in plain English"*. Or start a new session in the same folder and type `/setup` again: it picks up where it left off.

Saying yes (or no) to Claude

Claude often pauses to ask permission, sometimes in words like *Bash* or *git commit*. That's normal.

- **Allow it** if it matches what you just asked for, or what Claude just told you it was about to do.
- **Say no**, or ask *"what does that do, in plain English?"*, if it's a surprise, especially anything about deleting, sending, publishing or money.

Coming back the next day

1. Claude app → **Code** → **+ New session**.
2. Choose your project folder (it'll be in the recent list), Local, Manual.
3. *In a terminal instead:* open one in your project folder and type `claude`, `gemini` or `codex`.
4. Say **hello** (or `/start`). Check the 📁 line names the right folder.
5. Claude offers one next step. Say **"go"**, **"something else"**, **"I've got longer"** or **"low day"**.

Closed the window without finishing? Been away for weeks? Nothing is lost, and Claude doesn't make a thing of it. It catches up quietly and offers a small way back in.

Short sessions, long sessions, low days

Sessions can be as short or as long as you like. Claude doesn't ask you to commit to a length. It learns how you tend to work from how your sessions actually go, and sizes the next step to fit. When you're on a roll, it offers the next step. When you've done one and want to stop, stop.

On a day when you've got nothing in you, say **"low day"**. Claude offers three tiny things: talk for two minutes about what's on your mind, just open the file and read it, or one easy step. Or close it: the next step is written down for whenever you're back. There are no streaks and no "days since". It only keeps a record of what got done.

Talk, don't type

The biggest tip in this guide. Talk, using dictation, for almost everything you say to Claude. Most people speak about three times faster than they type, so you give Claude far more to go on, and get much better work back. **Be as messy as you like**: ramble, change your mind halfway through, say "um, no, actually…". Claude untangles it.

- **Use a proper dictation app rather than your computer's built-in one.** The built-in ones cut out after a pause and aren't made for fast, rambling talk. A good app lets you hold a key, talk, let go, and the text appears.
- **Wispr Flow** (*wisprflow.ai*) is the easiest start in the Claude app: free for 2,000 words a week on a computer. **Handy** (*handy.computer*) is free and turns your voice into text on your own computer. In a terminal, Claude Code has its own. See "Dictation apps" in Part 2, including what to switch off.
- **Can't install apps on a work laptop?** The built-in one will do to start: on a Mac, System Settings → Keyboard → Dictation; on Windows, `Windows key + H`.
- **Spell out names, numbers and dates**, or type them. Dictation mangles these, and Claude can't always guess.

Everyday use

- **Just ask**, in your own words: "Draft this week's specials board." "Write a polite chase-up to the flour supplier."
- **New material:** open your project folder, then `inbox`, and **copy** files in (right-click → Copy, then Paste) so your original stays put. Then say "process the inbox". Claude reads everything, files it, keeps the originals in an archive, and tells you what it did. **An empty inbox means everything's been dealt with.**
- **Finished documents:** Claude's drafts are plain text. Ask it to *"show me the draft"* to read it in the window, *"make a Word version"* for a document, or *"give me the text to paste into an email"*.
- **A thought on the go:** say *"jot this: …"* and Claude notes it in one line and carries on.
- **Stopping:** say "that's it for today". Claude writes down the next step and saves a snapshot, asking you nothing. If you just close the window instead, that's fine too.

Almost everything works by just asking. Two commands only work when **typed**: `/setup` and `/red-team`.

Four ground rules (already built in)

Claude is told to follow these every session. It usually does, but keep an eye on it.

1. **Everything Claude writes is a draft until you've checked it, and you answer for every word that goes out under your name.** If your work has rules about AI (a university, exam board, publisher or employer), those come first. Setup asks about them.
2. **Never invent.** AI can sound sure and be wrong. Claude says what it *checked* (and where), what it *thinks*, and what's *not known*, and never cites a source from memory. Check anything that matters.
3. **Anything Claude reads is sent to Anthropic**, the company that makes Claude. Keep passwords and keys out of your files (a tool's key goes in a `.env` file you type yourself), and keep very sensitive material outside the project folder. If this is for work or study, check your employer's or institution's AI policy.

4. **Ask before anything hard to undo**: deleting, sending, publishing, spending money, or throwing away unsaved work.

If something goes wrong

- **Talking about the wrong project?** Ask "which folder are you working in?" If it's wrong, start a new session and choose the right one.
- **Changed something you didn't want?** Say "undo that". Claude can often undo its own recent edits, and can always go back to the last snapshot.
- **Sounds too sure of itself?** Ask "did you check that, or is that a guess?"
- **An error message?** Say "explain that in plain English and tell me what to do".
- **Reached your limit?** The message says when your allowance comes back; until then Claude can't reply, whatever you change. When you start again, choose **Sonnet** from the **Model** dropdown next to the send button (it also shows which model you're on), and say "make my allowance last longer": Claude offers to save Sonnet as your default for all your projects. For one big piece of work you can switch back in the dropdown. On Pro, some models there are paid for with usage credits: check before you pick one.

PART 2 · WHEN YOU'RE COMFORTABLE

Part 1 is all you need to get going. Come back to this when you want more.

A second project

Give each piece of work **its own project folder**, so notes don't get muddled and Claude only reads what's relevant. **The test:** a different finished product or a different audience means a new folder, even if it's related. A bakery's day-to-day and its cookbook are two projects.

```
Claude projects
├── Bakery
├── Cookbook
└── Family history
```

In any project, say **"new project"**. Claude asks what it is, makes the folder next to the others, offers to copy anything across ("bring my recipes and the style guide"), and tells you how to open it: **+ New session** → choose the new folder → Local, Manual → type `/setup`. It already knows you, so it only asks about the new project.

You can also open a session on **Claude projects** and paste the sentence from Step 3 again.

You *can* keep everything in one folder; it works. It just gets cluttered.

About you, and about the work

Claude keeps one file **about you** that every project shares (what you do, how you like to work), and separate notes **for each project**. It decides which things go where and tells you; you don't need to manage it. If something really matters, say "remember that for all my projects" or "…for this project".

Skills: Claude remembers how

A skill is a saved set of instructions, like `/start`. The kit comes with thirteen, and you can make more. **You don't have to think of them yourself.** When you've asked for the same thing twice, or something works brilliantly, Claude offers, in one line:

```
Want me to remember how to do this, so next time you can just say
"chase the supplier"?
```

Say yes and it's done. Claude decides whether it's for this project or all of them. Say no and it won't ask again. You can also ask outright: "can you remember how to do that?"

Checking the setup fits

The kit starts the same for everyone. After your first good session, Claude mentions that you can **check how the setup fits what you're doing**. Say *"check the fit"*, then or any time. Another Claude, which hasn't seen your chats, reads your project's notes and suggests at most **three small changes**, such as a folder for each family line or something to check before saving. Claude explains each one in plain words and makes it only if you say yes. It never loosens the ground rules. Unlike `/red-team`, which checks a piece of *work*, this checks the *setup*. Say *"the full check"* for three critics instead of one; each critic is a separate run, so it uses noticeably more of your allowance.

Meetings

Got a meeting with a supervisor, manager, client or editor? Say so. Claude builds a one-page update from what you've done since last time, with your questions in your words. Afterwards, dictate your notes: Claude files them faithfully, puts actions and dates in your status, and records anything agreed.

Assessed work (students)

If setup hears that your work is marked or assessed, it adds a **study pack**. It turns the usual arrangement round: **you write, and Claude teaches and questions.**

- **Your university's rules come first.** Policies differ between universities and between modules. Your first step is to put your module's AI rules into `notes/ai-policy.md`, and to show your supervisor the "Rules for this work" section of `CLAUDE.md` and the full rules in `extras/study-pack/claude-section.md`, ideally at your next meeting.
- **Claude won't write assessed text or project code for you to submit.** It explains, asks questions, reviews what you've written, and helps you find bugs in your own code. Dictation for your report comes back as a faithful transcript, not tidied prose. If you insist, it's your call, and it's logged.
- **Everything is logged** in `notes/ai-use-log.md`, so your AI-use declaration is easy to write and accurate. Your git history, decisions log and experiment log show how the work developed.
- **Viva practice:** say "quiz me on this". Claude asks examiner-style questions, one at a time. Assessed work isn't signed off until you can explain it.
- **Honest references:** a sources register where only *you* mark a paper as read, and nothing unread gets cited.
- **For code and ML projects:** an experiment log with predictions written *before* each run, numbers only ever read from output files, a rule that the test set is touched once, and a red-team "evaluation auditor" that checks for data leakage and missing baselines.
- **If things get hard,** Claude will occasionally and briefly point you to a specific, practical route, like a two-line email to your supervisor or your university's support services. It won't nag, diagnose, or count the days.

Using it at work

- **Your employer's rules come first.** Setup asks whether an employer or client has rules about AI; paste them in and they're saved as `notes/ai-policy.md`, and Claude follows them.
- **Use the account your employer approves.** On a personal Free, Pro or Max account, Anthropic may use your chats and code to improve its models if that setting is on, and keeps them for up to five years if it is (30 days if not). Under commercial terms (Team, Enterprise, or the API), it doesn't train on them unless your organisation has chosen to share data. In a work folder, don't send feedback reports or session transcripts to Anthropic unless your employer allows it. If your organisation manages Claude Code's settings, its settings win wherever the two disagree, and its rules are added to the kit's.
- **The kit is public; your project folders aren't.** Nothing in the kit is confidential, but what you put in a project folder may be. Keep work folders on work equipment, and never push one anywhere public.

Building software

Building an app or a website from a brief, a requirements document or a list of stories? The **build pack** keeps the list of what it must do, plans one stage at a time, and checks each stage against the list, with things for you to try yourself. Say *"set up the build pack"*; the details are in Part 3, "Building software from a brief or a backlog".

Diagrams and pictures

Say *"draw a diagram of…"*: a flowchart, a timeline, a small family tree, an org chart. Claude draws it as text it can change later (Mermaid or SVG) and makes a preview page. Click its name in the chat to see it; it has buttons to save it as a PNG or SVG picture. It's free, with nothing to sign up for.

AI-made pictures (illustrations, mock-ups) are optional and **cost money**: from a few US cents to tens of cents each, depending on the model and size, with Google's Gemini image models, through a Google AI Studio account with billing switched on. Say *"set up pictures"* and Claude explains it first, and can help you set a monthly spending cap if you want one. You paste your key into a file yourself; Claude never opens it. You choose how often Claude checks the cost with you: before every picture (a good way to start), only above an amount you set, or never; change your mind any time by saying so. Each one is logged, carries Google's invisible watermark, and should be labelled as AI-made if you publish it.

Polishing work before it goes out

Say	What happens
"make it plainer"	**De-jargoning.** Claude lists hard words, long sentences and assumed knowledge, then rewrites in plain English, without making anything sound more certain than it is.
`/red-team` *(typed)*	**Red-teaming** means attacking your own work before anyone else does. Claude sends it to three separate critics who haven't seen your conversation (say, a regular customer, a fussy chef and a fact-checker), sifts their points, and gives you one list: must fix, should fix, optional. It uses a fair bit of your allowance, so save it for things that matter.
"check the fit"	**Tailoring.** A fresh-eyes check of how the setup suits your work, with at most three small changes, each only if you say yes.
"let's sign things off"	Claude walks you through work waiting for your approval, one item at a time, showing exactly what needs checking.

Ideas and decisions

- **"I've had an idea…"**: Claude writes it up with the honest case *against* it and a test that would settle it, so a good-sounding idea doesn't slip into finished work untested.
- **"Let's go with…"**: Claude records the decision and why. If it replaces an older one, the old one is marked "replaced", never deleted, so you can see why you changed your mind.

Dictation apps

Your computer's built-in dictation is a last resort: it cuts out after a pause and isn't made for fast, natural talk. Any of these is better. Each one works by holding a key while you talk.

App	What it is
Claude Code's own `/voice`	Built into Claude Code in a terminal: type `/voice` once, then hold Space and talk. From then on, holding Space records you instead of typing spaces. Nothing to install, and it doesn't use your Claude allowance. Needs a Claude account sign-in. Your audio goes to Anthropic. Not in the Claude app.
Wispr Flow *wisprflow.ai*	The most polished: it tidies away the "ums" and works in any app, terminals included. Mac, Windows, iPhone, Android. Free for 2,000 words a week on a computer (phones have their own limit); paid for unlimited. Works online, and by default it may learn from your recordings: in its settings, turn on Privacy Mode.
Handy *handy.computer*	Free. Turns your voice into text entirely on your computer, so no recording goes anywhere. Mac, Windows, Linux. Plain transcription, without the tidying, which Claude doesn't need anyway.
Superwhisper *superwhisper.com*	Mac, Windows, iPhone. A free version runs on your computer; the paid one adds more models, some in the cloud.

Whichever you use, the words themselves still reach Claude (or ChatGPT, or Gemini) as text, so check your school's or employer's rules before talking about pupils, patients or clients. **Codex**'s `/voice` is a spoken conversation, not dictation, so use one of the apps. **Gemini CLI** has an experimental voice mode you have to switch on; an app is simpler. Prices and limits change: check the app's own site.

What's in a project folder

```
status.md     Where things stand: current goal, next steps, dates.
handover.md   The next step, ready for when you're back.
done.md       Everything finished. Nothing is ever removed.
review-queue.md  Work waiting for your sign-off.
inbox         Put things here unsorted. Claude sorts them and empties it.
notes         What Claude knows: style guide, glossary, open questions.
ideas         Untested ideas, with the case against.
decisions     What you decided, why, and what it replaced.
drafts        Work in progress.
finished      Work you've approved.
private       Confidential material and archived originals. Not in snapshots.
              (Appears the first time the inbox is sorted.)
CLAUDE.md     Claude's standing orders for this project (read every session).
```

Some setup files are hidden on purpose. You never need to touch them.

Snapshots and backups

- **Snapshots** are saved every time you finish a session and every time you sign something off. Ask in plain English: "what changed last week?", "show me the menu as it was on Tuesday".
- **Not in snapshots:** the `private` folder, including the archive of your inbox originals. **Back up your whole Claude projects folder yourself**: Time Machine on a Mac, File History on Windows, or a copy on an external drive.

PART 3 · FOR THE TECHNICALLY MINDED

If you already use Claude Code

- When you accept the "About me" extra (offered after a session that went well), Claude adds a section to `~/.claude/CLAUDE.md`, which applies in **every** project, including existing repos. It shows you the text first and never replaces existing content.
- A personal skill in `~/.claude/skills/` overrides a project skill of the same name. Claude checks for clashes with the kit's skills when it sets up "About me", and offers to rename.
- The kit is kept as a git clone in `~/.claude/workspace-kit/`; its `claude-workspace` folder is the blank template. "New project" updates it (`git pull --ff-only`) and copies only that folder, so every project starts with its own fresh history. Updating the kit never changes projects you already have.

If you write code

- **Permission mode:** once you trust it, **Accept edits** mode cuts the prompts a lot. The kit already lets Claude save snapshots and update its own notes without asking, and still asks before `git push`, `rm` and anything that throws work away.
- **Big files and keys:** before every snapshot, Claude checks for files over 20 MB, folders of hundreds of new files (a dataset, say), and API keys written into code or notebooks. None of those get saved. A key that was ever committed or pasted into chat must be cancelled with the provider and replaced.
- **Existing project?** Don't copy the kit over it. Open your project and say: *"Bring in the workspace kit from ~/.claude/workspace-kit/claude-workspace (clone github.com/teedor/ai-workspace-kit there first if it's missing). Don't overwrite anything: merge CLAUDE.md, append to .gitignore, show me the changes first."* Then `/setup`.
- **Code in other repositories?** Say "link a repo". The linked-repos pack keeps a register (`repos.md`) of each repository's path, remote and mode: **read-only** unless you say otherwise. Repositories stay **outside** the workspace (a clone goes in `~/Linked repos/`; an existing checkout stays put) and are reached through `permissions.additionalDirectories` in your local settings, which grants file access only: their own skills and settings don't load, and their code never lands in the workspace's snapshots. Claude reaches them with `git -C`, which always asks first. Commits, branches and pull requests happen in that repository, its way, and nothing is pushed without your yes.
- **One person per folder.** In a team repo, keep personal rules in `CLAUDE.local.md`, and agree with the team before committing workspace files.
- Code keeps its own layout. "Handover" commits only workspace files. "Sign off" runs your tests, commits reviewed code, and asks before committing to `main`. The red team's checker runs code only against test data.
- `.claude/settings.json` blocks Claude's file tools from reading `.env` and `.pem` files (including `.env.example`) and other common key and credential files, and makes it ask before `git push`, `rm`, `git reset`, `git checkout` and similar, and before any `git -C` command (git run in another

folder), in Bash and PowerShell. It sets the starting permission mode to asking. These are guard rails, not a sandbox.

- `.claude/agents/critic.md` is a sub-agent limited to reading (Read, Grep, Glob). `/tailor` uses it for every critic, `/red-team` for critics that don't need to run anything, and the build pack to map code and check plans and stages.

Building software from a brief or a backlog

Say *"set up the build pack"*, or drop a brief or a board export in the inbox of a project with code. It gives you an overall plan without losing the freedom to change direction: **the list** of what the software must do is the big picture, and only **the next stage** is planned in detail.

- **The list** (`notes/requirements.md`) comes from a requirements document, from stories exported from a board, or from talking it through. From Azure DevOps: run a query, then **Export to CSV**. From GitHub Projects: the view's menu, then **Export view data** (a `.tsv` file). Export a slice (one epic, this sprint, what's not done), and leave out people columns ("Assigned to", "Assignees", "Created by") and customer details. The board stays the master: its own IDs are kept, and exporting again shows you what's new, changed or gone before anything changes. Claude never changes items on the board.
- **The route:** one line per stage, in order, each named by what you'll be able to do. `/stage` plans the next one, asking you about how things should work, not about technology. Changes of direction are recorded with `/decide`, and nothing on the list is dropped or shrunk without your yes.
- **Each task has a check**, said in plain words before the code is written, and seen to fail and then pass. Nothing runs until you've said it's safe (tests and apps can touch a real database, send real emails or take real payments), and only on made-up data. If you don't know, Claude can set things up so it's safe.
- `/check-stage` has a fresh critic compare the code with the list, not with the task list, and hunt for checks that only look like checks. Then you try the main things yourself, and `/signoff` commits it (in a linked repository, sign-off leads to the pull request). The two fresh-eyes checks use a little more of your allowance than ordinary work.
- **No branches inside a workspace** (the notes would go with them). A team's repository is always linked (read-write), never used as the workspace itself: each stage is then usually one story, on its own branch, ending in a pull request, the repository's way. Going live always asks. A commit message that names a board item can link to it once pushed, so board IDs stay out of commit messages unless the repository's convention uses them, and "fixes" never goes next to one, because merging can close the item.
- Not for assessed work. For dozens of requirements across a large codebase or a team, a tool built for planning large projects from a written specification may suit you better.

Prefer the terminal?

The app's Code tab and the terminal are the same Claude Code: same folders, skills and rules.

1. **Open a terminal.** Mac: `Cmd + Space`, type `Terminal`, press Enter. Windows: `Windows key + X`, then **Terminal** (Windows 11) or **Windows PowerShell** (Windows 10). Linux: your usual terminal; use the Mac command below.
2. **Install (once):**

```
Mac:      curl -fsSL https://claude.ai/install.sh | bash
Windows:  irm https://claude.ai/install.ps1 | iex
```

Open a new window and type `claude --version` to check it worked.

3. **Go to your folder**: **Claude projects** the first time (to paste the sentence from Step 3), your project folder after that. Mac: type `cd` and a space, drag the folder into the window, and press Enter. Windows: open the folder in File Explorer, type `powershell` in the address bar, and press Enter.
4. Type `claude`. The first time, it opens your browser to sign in.

Recent versions start the terminal in **auto** mode. The kit's settings ask for the asking mode instead; check the mode shown at the bottom, and press `Shift + Tab` to change it. Claude Code has its own dictation in the terminal: type `/voice`, then hold Space and talk (see "Dictation apps" in Part 2).

Claude Code has built-in commands such as `/resume`, so the kit's skills are named `/start` and `/signoff` to avoid clashes.

Using Gemini CLI instead

The kit also works with Google's **Gemini CLI**, in a terminal. Each project folder carries a small adapter (`GEMINI.md`, a `.gemini` folder and `.geminiignore`), so the same rules and the same commands (`/start`, `/setup`, `/handover`…) work there.

- **Install:** on a Mac with Homebrew, `brew install gemini-cli`; otherwise install Node.js 20 or later from *nodejs.org*, then `npm install -g @google/gemini-cli`. On Windows you also need Git for Windows, as in Step 2.
- **Start:** open a terminal in your **Claude projects** folder (see "Prefer the terminal?" above), type `gemini`, choose **Sign in with Google** with a personal account (school and work accounts need extra Google Cloud set-up), and trust the folder when asked. Then paste the sentence from Step 3. In your project folder later, choose **Acknowledge and Enable** when Gemini asks about new agents (that's the kit's read-only critic).
- **Privacy:** on a free personal account, Google may use your prompts to improve its products, and people may read them. Type `/privacy` to choose.
- **What's different:** the kit's safety settings are Claude Code's, so with Gemini they're rules it's told to follow rather than locks. Gemini still asks before changing files or running commands. The linked repos pack is Claude-only for now.

Using Codex (ChatGPT) instead

If you have a ChatGPT plan, OpenAI's **Codex** works too, in a terminal. Each project folder carries a small adapter for it (`AGENTS.md` and a `.codex` folder).

- **Install:** Mac: `curl -fsSL https://chatgpt.com/codex/install.sh | sh`, or `brew install --cask codex`. Windows, in a terminal (see "Prefer the terminal?"): `powershell -ExecutionPolicy ByPass -c "irm https://chatgpt.com/codex/install.ps1 | iex"`.

- **Start:** open a terminal in your **Claude projects** folder, type `codex`, choose **Sign in with ChatGPT** (Codex comes with Plus, Pro, Business, Edu and Enterprise plans), and **Trust and continue** when asked. In ChatGPT's settings, check whether OpenAI may use your conversations to improve its models, and turn that off if you'd rather it didn't. Then paste the sentence from Step 3.
- **One difference to remember:** commands start with `$`, not `/`: `$setup`, `$start`, `$handover`.
- **What's different:** inside the project folder, Codex edits files and runs most commands without asking, but saving snapshots asks each time (expected: Codex protects the snapshot folder). On Windows, after you trust the folder, Codex offers to set up its sandbox: choose **Set up default sandbox** and say yes to the Windows administrator box. The kit adds rules so it always asks before deleting, pushing, or anything that could throw work away, and asks before stepping outside the folder; the rest of the kit's safety settings are rules it's told to follow rather than locks. Check your ChatGPT data settings before bringing in other people's details. The linked repos pack is Claude-only for now.

Off-computer backup with GitHub

Optional. Make a free **private** repository on github.com and ask Claude to "back this up to GitHub". Sign in through your browser when asked, and never paste passwords or tokens into the chat. Your name and email go with every snapshot (GitHub offers a private "noreply" email for this), and anything once saved stays in the history, even if deleted later. The `private` and `inbox` folders are never included.